Appello 14 settembre 2015

Politecnico di Milano

Anno accademico 2014-2015

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:

Esercizio 1

Si consideri il seguente metodo che conta le occorrenze della stringa `param` in `testo`:

```
static ArrayList<Integer> trovaOccorrenze(String testo, String param)
```

Si scrivano le pre- e post-condizioni JML del metodo ipotizzando che:

1. Il metodo restituisce sempre un `ArrayList` di un elemento. Se il metodo trova almeno un'occorrenza, il primo elemento della lista contiene la posizione in `testo` del primo carattere di `param`. Se il metodo non trova occorrenze, il primo elemento della lista vale -1;

2. Il metodo restituisce un `ArrayList` con un numero di elementi variabile. Se la lista non è vuota, i suoi elementi contengono le posizioni in ordine crescente (primo carattere) di tutte e sole le occorrenze di `param` in `testo`. Se il metodo non trova occorrenze, la lista contiene solo il valore -1;

3. l'intestazione del metodo contenga un terzo parametro `int prec`, ovvero la precisione da usarsi per la ricerca delle occorrenze. Un'occorrenza di `param` in `testo` si considera valida se è una sottostringa di `testo` che ha la stessa lunghezza di `param` e il numero dei caratteri diversi, nelle stesse posizioni, è al piu' uguale a `prec`.
   Ad esempio se `testo` valesse `cassa` e prec 3, valori accettati di `param` sono per esempio `colla`, `carta`, `masso`.

SOLUZIONE:
Usiamo per comodita' i metodi indexOf della classe String, cosi' definiti:
`public int indexOf(String str)`: restituisce l'indice della prima comparsa della string str nella stringa this, oppure -1 se la stringa str non compare.
`public int indexOf(String str, int fromIndex)`: restituisce l'indice della prima comparsa della string str nella stringa this partendo dall'indice fromIndex, oppure -1 se la stringa str non compare.

1.

```
//@requires testo!=null && param!=null
//@ensures \result.get(0).equals(testo.indexOf(param,0))
static ArrayList<Integer> trovaOccorrenze(String testo, String param)
```

2.

```
//@requires testo!=null && param!=null
//@ensures
(\forall int y;;
    (\result.contains(y) <==>
        (
            (testo.indexOf(param, y)==y &&  0<=y<testo.length())
            ||
            (y==-1 && testo.indexOf(param,0)==-1)
         )
    )
) &&
// result e' ordinato in modo crescente
(\forall int i; 0<=i && i<\result.size()-1;
    \result.get(i)<\result.get(i+1));
static ArrayList<Integer> trovaOccorrenze(String testo, String param)
```

3.

```
//@requires testo!=null && param!=null
//@ensures
(\forall int i; ;
   \result.contains(i) <==>
```

```
    // la stringa corrisponde
    (
        (\num_of int j; 0<=j && j<param.length();
                testo.charAt(i+j)<param.charAt(j)) <=prec
        &&
        0<=i && i<testo.length()-param.length()
        ||
        (
            i==-1 &&
            !(\exists int k; 0<=k && k<testo.length()-param.length();
                   (\num_of int j; 0<=j && j<param.length();
                testo.charAt(k+j)<param.charAt(k)) <=prec
            )
         )
    )
)&&
// result e' ordinato in modo crescente
(\forall int i; 0<=i && i<\result.size()-1;
    \result.get(i)<\result.get(i+1));
static ArrayList<Integer> trovaOccorrenze(String testo, String param)
```

Esercizio 2

Date le classi Java `Volt` e `PresaCorrente`, che restituisce corrente a 220 volt:

```
public class Volt {
   private int volts;

   public Volt(int v) {
      this.volts=v;
   }

   public int getVolts() {
      return volts;
   }

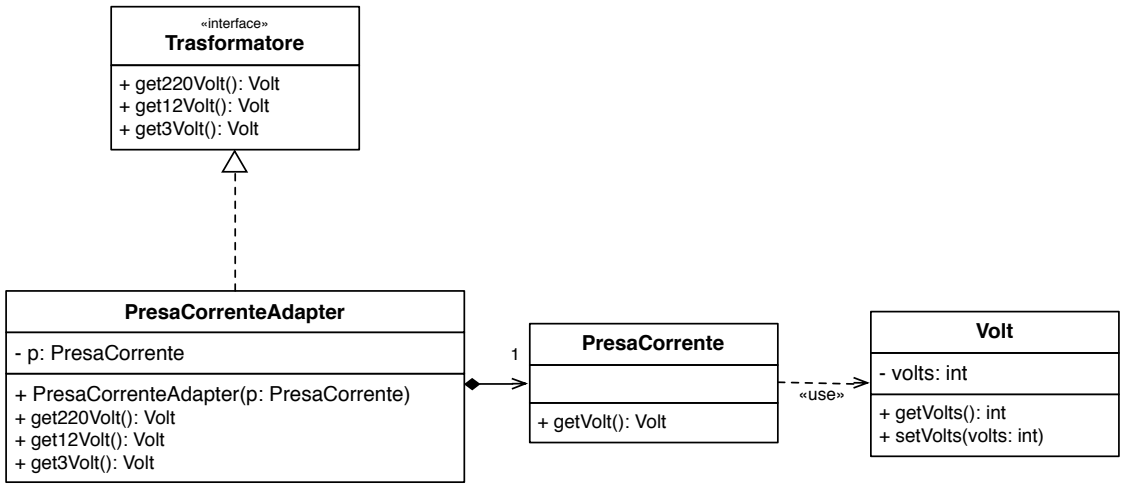
   public void setVolts(int volts) {
      this.volts = volts;
   }
}

public class PresaCorrente {
   public Volt getVolt(){
      return new Volt(220);
   }
}
```

Si utilizzi il design pattern **Adapter** per "realizzare" un trasformatore capace di erogare corrente a 3, 12, e 220 volt:

```
public interface Trasformatore {
   public Volt get220Volt();
   public Volt get12Volt();
   public Volt get3Volt();
}
```

Si definisca la classe Java da aggiungere, specificando anche le relazioni con le classi esistenti.



SOLUZIONE:

```
public class PresaCorrenteAdapter implements Trasformatore {
```

```
    private final PresaCorrente presaCorrente;

    public PresaCorrenteAdapter(PresaCorrente presaCorrente) {
        super();
        this.presaCorrente = presaCorrente;
    }

    private Volt convertTo(int value) {
        Volt result = presaCorrente.getVolt();
        result.setVolts(value);
        return result;
    }

    @Override
    public Volt get220Volt() {
        return convertTo(220);
    }

    @Override
    public Volt get12Volt() {
        return convertTo(12);
    }

    @Override
    public Volt get3Volt() {
        return convertTo(3);
    }
}
```

Esercizio 3

Un gruppo di 10 amici ha a disposizione una botte con 100 litri di vino e 2 rubinetti per berli. Di conseguenza possono bere solo due persone alla volta, sempre che la botte non sia vuota. Si supponga, per semplicità, che ogni persona beve sempre $1/2$ litro di vino. Si scriva un programma (concorrente) Java che simuli la situazione appena descritta. Il thread `Bevitore`, dopo aver atteso un tempo casuale, deciderà di bere e si metterà in attesa se il numero di rubinetti liberi fosse 0. Dopo aver ottenuto la disponibilità di un rubinetto, controllerà la quantità di vino rimasta e, se possibile, completerà la bevuta in sorsate di 10ml. Se la botte fosse vuota, il `Bevitore` terminerà la sua esecuzione.

La soluzione deve prevedere:

- una classe `Botte`, con un metodo `bevi()` (usabile in maniera concorrente) che consente di decrementare il contenuto della botte di 1/2 litro;
- una classe `Gestore`, che fornisce due metodi (usabili in maniera concorrente) `occupaRubinetto()` e `rilasciaRubinetto()` che consentono di acquisire uno dei due rubinetti, se libero, e rilasciarlo dopo la bevuta,

si suppone che ogni amico che vuole bere dalla botte esegua sempre la sequenza di operazioni:

- `gestore.occupaRubinetto();`
- `botte.bevi();`
- `gestore.rilasciaRubinetto();`

Si devono pertanto implementare le classi `Gestore` e `Botte`.

SOLUZIONE:

```
public class Botte {
    private int mlRimasti;

    public Botte(){
        this.mlRimasti=100000;
    }
    public int bevi() throws InterruptedException{
        int effettivamenteBevuto=0;
        int sorseggio=10;
        while(effettivamenteBevuto<500 && !isEmpty()){
         effettivamenteBevuto=effettivamenteBevuto+this.sorseggia(sorseggio);
        }
        return effettivamenteBevuto;
   }
    private synchronized boolean isEmpty(){
        return this.mlRimasti==0;
   }
    private synchronized int sorseggia(int quantita){
        if(mlRimasti>quantita){
            this.mlRimasti=this.mlRimasti-quantita;
            return quantita;
       }
       else{
           int bevuta=this.mlRimasti;
           this.mlRimasti=0;
           return bevuta;
       }
```

```
    }
}


public class GestoreRubinetti {
    private int rubinettiLiberi;

    public GestoreRubinetti(){
        this.rubinettiLiberi=2;
    }
    public synchronized void occupaRubinetto() throws InterruptedException{
        while(rubinettiLiberi==0){
            wait();
        }
        rubinettiLiberi--;
    }
    public synchronized void rilasciaRubinetto(){
        rubinettiLiberi++;
        notify();
    }
}
```

Esercizio 4

Si consideri il metodo Java seguente:

```
public static int foo(int n) {
   int t=0;

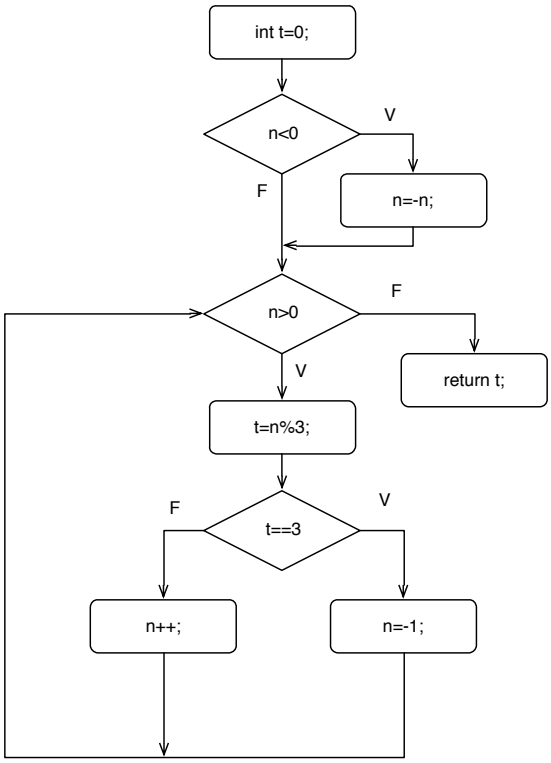
   if (n < 0) n = -n;

   while (n >0) {
      t = n%3;
      if (t == 3) n = -1;
      else n++;
   }
   return t;
}
```

e si definisca

1. Il diagramma del flusso di controllo.
2. Un insieme minimo di test che copra tutte le istruzioni. Nel caso non fosse possibile, definire la copertura (in percentuale) dell'insieme di test definiti.
3. La condizione che n deve rispettare per eseguire il programma senza entrare nel ciclo *while*. È possibile? Cosa restituirebbe il programma?
4. Si modifichi il test dell'istruzione if da `t==3` in `t==2` e si calcoli la precondizione che n dovrebbe rispettare affinche' si entri nel ciclo due volte e la prima volta si esegua il ramo then mentre la seconda si esegua il ramo else. Calcolare anche un caso di test, se esiste, che soddisfa la precondizione.

SOLUZIONE: 1.



2.

`n=-1`

Copre tutte le istruzioni tranne il ramo then del secondo if. Infatti, all'ingresso del ciclo `while`, n=1. Ad ogni iterazione il valore di `n` viene sempre incrementato finchè il valore di `n` supera il massimo valore contenibile in un intero, e di conseguenza per overflow, il valore di `n` diviene negativo. A quel punto si esce dal ciclo `while`.

Copertura 8/9.

3.

n=0.

ritorna t=0;

4.

$$\begin{cases} n \neq 0 \\ |n|\%3 = 2 \\ -1 > 0 \\ -1\%3 \neq 2 \\ 0 \leq 0 \end{cases}$$

Non è possibile coprire il suddetto cammino in quanto $-1 \geq 0$ non può mai essere soddisfatta.